

HỌC MÁY CƠ BẢN VỚI PYTHON

Chương 2 – Buổi 2

pandas, trực quan hóa và scikit-learn

(pandas, Visualization & scikit-learn)

Lê Nhật Tùng

Bài giảng đi kèm giáo trình

Mục tiêu buổi học

Sau buổi học này, bạn có thể:

- Đọc, xem, lọc, thêm cột, sửa và tổng hợp **dữ liệu dạng bảng** (*tabular data*) bằng **pandas**.
- Vẽ các biểu đồ cơ bản bằng **Matplotlib** và **Seaborn**; tính tay và đọc được **biểu đồ hộp** (*box plot*).
- Biết chọn loại biểu đồ theo mục đích.
- Hiểu cách gọi chung của mọi mô hình trong `scikit-learn`: `fit`, `predict`, `score`, `transform`.

Nội dung

1. pandas: dữ liệu dạng bảng

2. Trực quan hóa dữ liệu

3. Làm quen với scikit-learn

4. Thực hành và tổng kết

Phần 1

pandas: dữ liệu dạng bảng

Chuẩn bị: nạp lại thư viện cho Buổi 2.2

ch02.py — ô chuan_bi_buoi2

```python
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
pd.set_option("display.max_columns", None)
pd.set_option("display.width", 85)

def xep_loai(d):
    """Trả về xếp loại học lực từ điểm số d (đã viết ở Buổi 2.1)."""
    if d >= 8.5:
        return "Giỏi"
    elif d >= 7.0:
        return "Khá"
    elif d >= 5.0:
        return "Trung bình"
    return "Yếu"
```

Ô chuẩn bị nạp lại bốn thư viện và định nghĩa lại hàm `xep_loai` của Buổi 2.1 – notebook Buổi 2.2 chạy được độc lập.

- Mảng NumPy chỉ chứa **một kiểu** dữ liệu và **không có tên cột** $\Rightarrow$ bất tiện với dữ liệu thật (có cột chữ, có ô trống).
- pandas giải quyết bằng hai cấu trúc: **Series** (một cột có nhãn) và **DataFrame** (bảng).

Series: một cột dữ liệu có nhãn

Dữ liệu thực tế hiếm khi chỉ toàn số: có tên cột, có cột chữ (giới tính, thành phố), có ô bị trống. pandas cung cấp hai cấu trúc chính:

- **Series**: **một cột** dữ liệu có nhãn (chỉ mục).
- **DataFrame**: **một bảng** gồm nhiều cột, mỗi cột là một Series.

ch02.py — ô pd_series

```
1 s = pd.Series([7.5, 8.0, 6.5], index=["An", "Bình",
    ↪  "Chi"], name="diem")
2 print(s)
3 print("Điểm của Bình:", s["Bình"])
```

Kết quả

```
An      7.5
Bình    8.0
Chi     6.5
Name: diem, dtype: float64
Điểm của Bình: 8.0
```

Quy ước: `import pandas as pd`.

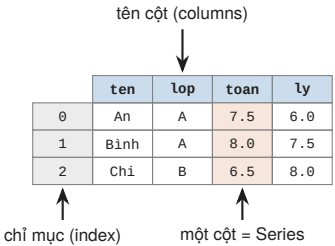
DataFrame: bảng dữ liệu

ch02.py — ô pd_dataframe

```
bang = pd.DataFrame({
    "ten":   ["An", "Bình", "Chi", "Dũng", "Hà", "Khoa"],
    "lop":   ["A", "A", "B", "B", "A", "B"],
    "toan":  [7.5, 8.0, 6.5, 9.0, 5.5, 8.5],
    "ly":    [6.0, 7.5, 8.0, 8.5, 6.5, 7.0],
    "gio_tu_hoc": [10, 12, 8, 15, 5, 11],
})
print(bang)
```

Kết quả

```
    ten lop  toan   ly  gio_tu_hoc
0    An   A   7.5  6.0          10
1  Bình   A   8.0  7.5          12
2   Chi   B   6.5  8.0           8
3  Dũng   B   9.0  8.5          15
4    Hà   A   5.5  6.5           5
5  Khoa   B   8.5  7.0          11
```



- **Chỉ mục** (*index*): nhãn của mỗi dòng (mặc định 0, 1, 2…).
- **Tên cột** (*columns*): nhãn của mỗi cột.
- Tạo DataFrame từ một **dict**: khóa = tên cột, giá trị = danh sách dữ liệu.

Ví dụ đời thường: DataFrame là một trang tính Excel

Ví dụ đời thường: Excel điều khiển bằng câu lệnh

Hàng tiêu đề là tên cột, cột ngoài cùng bên trái là số thứ tự dòng (*index*), mỗi ô chứa một giá trị. Khác biệt: thay vì bấm chuột, ta điều khiển bằng câu lệnh – nên lặp lại chính xác các thao tác trên bảng 1 triệu dòng, hay trên dữ liệu tháng sau, chỉ bằng cách chạy lại mã.

- Tạo DataFrame từ một **từ điển**: mỗi khóa là tên cột, mỗi giá trị là danh sách các ô (cùng độ dài).
- Không chỉ định chỉ mục $\Rightarrow$ pandas tự đánh số dòng 0, 1, 2…

Đọc và ghi file CSV

ch02.py — ô pd_csv

```python
bang.to_csv("diem_sinh_vien.csv", index=False)    # ghi ra file CSV
bang2 = pd.read_csv("diem_sinh_vien.csv")         # đọc lại từ file CSV
print(bang2.shape)
print(bang2.head(3))
```

Kết quả

```
(6, 5)
    ten lop  toan   ly  gio_tu_hoc
0    An   A   7.5  6.0          10
1  Bình   A   8.0  7.5          12
2   Chi   B   6.5  8.0           8
```

- **CSV** (*Comma-Separated Values*): file văn bản, mỗi dòng một bản ghi, các giá trị ngăn cách bằng dấu phẩy. Excel, Google Sheets đều xuất được CSV.
- `to_csv`: ghi bảng ra file; `index=False` để không ghi cột chỉ mục.
- `read_csv`: đọc file – kể cả từ một đường link trên Internet.

Mẹo nhỏ

Trên Colab: tải file lên bằng biểu tượng thư mục ở thanh bên trái, rồi `pd.read_csv("ten_file.csv")`.

Xem tổng quan: `info()`

ch02.py — ô pd_info

```
1  bang.info()
```

Kết quả

```
<class 'pandas.DataFrame'>
RangeIndex: 6 entries, 0 to 5
Data columns (total 5 columns):
 #   Column      Non-Null Count  Dtype
---  ------      --------------  -----
 0   ten         6 non-null      str
 1   lop         6 non-null      str
 2   toan        6 non-null      float64
 3   ly          6 non-null      float64
 4   gio_tu_hoc  6 non-null      int64
dtypes: float64(2), int64(1), str(2)
memory usage: 372.0 bytes
```

Mẹo nhỏ

Ba lệnh nên chạy **ngay khi có bảng mới**: `head()`, `info()`, `describe()`.

`info()` cho biết:

- số dòng, tên các cột;
- số ô **không trống** (*non-null*) của mỗi cột – phát hiện **giá trị thiếu** (*missing values*);
- **kiểu dữ liệu** (*dtype*): `float64`, `int64`, cột chữ hiện `str` (pandas 3) hoặc `object` (pandas 2, trên Colab) – cùng chỉ cột văn bản.

Thống kê nhanh các cột số: `describe()`

ch02.py — ô pd_describe

```
print(bang.describe().round(2))
```

Kết quả

```
       toan    ly  gio_tu_hoc
count  6.00  6.00        6.00
mean   7.50  7.25       10.17
std    1.30  0.94        3.43
min    5.50  6.00        5.00
25%    6.75  6.62        8.50
50%    7.75  7.25       10.50
75%    8.38  7.88       11.75
max    9.00  8.50       15.00
```

`describe()` tính cho mỗi cột số:

- `count`: số giá trị không trống;
- `mean`: **trung bình** (*mean*);
- `std`: **độ lệch chuẩn** (*standard deviation*);
- `min`, `max`: nhỏ nhất, lớn nhất;
- `25\%`, `50\%`, `75\%`: các **phân vị** (*percentiles*); `50\%` chính là **trung vị** (*median*).

Ý nghĩa các đại lượng này được giải thích kỹ ở Chương 3.

Bài tập mẫu: kiểm tra `describe()` bằng tay

Cột `toan`: 7,5; 8,0; 6,5; 9,0; 5,5; 8,5. Tính `count`, `mean`, `min`, `max`, `50\%`.

Bài tập mẫu

Lời giải.

- `count` = 6 (6 ô không trống).
- `mean` = $(7{,}5 + 8{,}0 + 6{,}5 + 9{,}0 + 5{,}5 + 8{,}5)/6 = 45/6 = 7{,}5$.
- Sắp xếp: 5,5; 6,5; 7,5; 8,0; 8,5; 9,0 $\Rightarrow$ `min` = 5,5; `max` = 9,0.
- **Trung vị** (*median*): 6 số (chẵn) $\Rightarrow$ trung bình hai số giữa $(7{,}5 + 8{,}0)/2 = 7{,}75$.

Trung vị (7,75) lớn hơn trung bình (7,5) vì điểm 5,5 của Hà kéo trung bình xuống.

Chọn cột

ch02.py — ô pd_chon_cot

```
print(bang["toan"].tolist())                # một cột -> Series
print(bang[["ten", "toan"]].head(3))        # nhiều cột -> DataFrame
```

Kết quả

```
[7.5, 8.0, 6.5, 9.0, 5.5, 8.5]
    ten  toan
0    An   7.5
1  Bình   8.0
2   Chi   6.5
```

Lưu ý

- **Một** cặp ngoặc vuông `bang["toan"]` → một **Series**.
- **Hai** cặp ngoặc `bang[["ten", "toan"]]` (một *danh sách* tên cột) → một **DataFrame**.

Khi tạo X cho `scikit-learn` với một cột, dùng `df[["cot"]]` để có bảng hai chiều.

Chọn dòng: `.loc` và `.iloc`

ch02.py — ô pd_loc_iloc

```
print(bang.loc[2, "ten"])                    # loc: theo NHÃN dòng/cột
print(bang.iloc[0:2, 0:3])                   # iloc: theo VỊ TRÍ (số thứ tự)
```

Kết quả

```
Chi
    ten lop  toan
0    An   A   7.5
1  Bình   A   8.0
```

- `.loc[nhãn_dòng, nhãn_cột]`: chọn theo **tên** (*label-based*).
- `.iloc[vị_trí_dòng, vị_trí_cột]`: chọn theo **số thứ tự** (*integer position*), giống NumPy.
- Với `.loc`, lát cắt `0:2` lấy **cả** nhãn 2; với `.iloc`, `0:2` **không** lấy vị trí 2.

`.loc` và `.iloc` khác nhau khi nào?

- `bang.loc[2, "ten"]`: dòng có **nhãn** 2, cột có **tên** `ten` $\Rightarrow$ Chi.
- `bang.iloc[0:2, 0:3]`: dòng ở **vị trí** 0 đến trước 2, cột ở vị trí 0 đến trước 3.
- Bảng vừa tạo có nhãn dòng trùng vị trí (0, 1, 2…) nên hai cách **trông giống nhau**.
- Sau khi sắp xếp theo điểm trung bình, dòng đầu tiên có nhãn 3 (Dũng):
  - `.iloc[0]` $\Rightarrow$ Dũng (vị trí đầu tiên);
  - `.loc[0]` $\Rightarrow$ vẫn là An (nhãn 0).

Lọc dòng theo điều kiện

ch02.py — ô pd_loc_dong

```
print(bang[bang["toan"] >= 8])                                  # lọc 1 điều kiện
print(bang[(bang["lop"] == "A") & (bang["ly"] > 6.0)])          # kết hợp: & (và), | (hoặc)
```

Kết quả

```
    ten lop  toan   ly  gio_tu_hoc
1  Bình   A   8.0  7.5          12
3  Dũng   B   9.0  8.5          15
5  Khoa   B   8.5  7.0          11
    ten lop  toan   ly  gio_tu_hoc
1  Bình   A   8.0  7.5          12
4    Hà   A   5.5  6.5           5
```

Lưu ý

Giống NumPy, ta dùng mặt nạ True/False. Kết hợp nhiều điều kiện bằng `&` (và), `|` (hoặc), `~` (phủ định) – **không** dùng `and`/`or` – và **bọc mỗi điều kiện trong ngoặc tròn**.

Bài tập mẫu: viết điều kiện lọc

Viết lệnh lọc các sinh viên (a) lớp B; (b) tự học từ 10 giờ **hoặc** toán trên 8,5; (c) lớp A **và** toán dưới 7.

Bài tập mẫu: viết điều kiện lọc

Viết lệnh lọc các sinh viên (a) lớp B; (b) tự học từ 10 giờ **hoặc** toán trên 8,5; (c) lớp A **và** toán dưới 7.

Bài tập mẫu

Lời giải.

- (a) `bang[bang["lop"] == "B"]` ⇒ Chi, Dũng, Khoa. So sánh bằng dùng **hai** dấu ==.
- (b) `bang[(bang["gio_tu_hoc"] >= 10) | (bang["toan"] > 8.5)]` ⇒ An, Bình, Dũng, Khoa.
- (c) `bang[(bang["lop"] == "A") & (bang["toan"] < 7)]` ⇒ chỉ có Hà (5,5).

Lưu ý

Mỗi điều kiện phải nằm trong **ngoặc tròn**: `&` được Python tính trước == và >.

Thêm cột, áp dụng hàm và sắp xếp

ch02.py — ô pd_them_cot

```
bang["tb"] = (bang["toan"] + bang["ly"]) / 2          # cột mới từ cột cũ
bang["xep_loai"] = bang["tb"].apply(xep_loai)         # áp dụng hàm cho từng giá trị
print(bang.sort_values("tb", ascending=False))
```

Kết quả

```
    ten lop  toan   ly  gio_tu_hoc    tb  xep_loai
3  Dũng   B   9.0  8.5          15  8.75     Giỏi
1  Bình   A   8.0  7.5          12  7.75      Khá
5  Khoa   B   8.5  7.0          11  7.75      Khá
2   Chi   B   6.5  8.0           8  7.25      Khá
0    An   A   7.5  6.0          10  6.75  Trung bình
4    Hà   A   5.5  6.5           5  6.00  Trung bình
```

- Gán `bang["tb"] = ...`: tạo **cột mới** từ các cột cũ (vector hóa, không cần vòng lặp).
- `.apply(xep_loai)`: gọi hàm `xep_loai` (đã viết ở phần ôn Python) cho **từng giá trị** của cột.
- `sort_values("tb", ascending=False)`: **sắp xếp** *(sort)* giảm dần theo cột `tb`.

Đổi tên, xóa cột và sửa giá trị

ch02.py — ô pd_sua_xoa

```
bang_moi = bang.rename(columns={"gio_tu_hoc": "gio"})   # đổi tên cột
bang_moi = bang_moi.drop(columns=["xep_loai"])           # xóa một cột
bang_moi.loc[4, "toan"] = 6.0                            # sửa một ô: dòng nhãn 4 (Hà)
print(bang_moi.head())
print("Bảng gốc vẫn giữ điểm toán của Hà:", bang.loc[4, "toan"])
```

Kết quả

```
    ten lop  toan   ly  gio    tb
0    An   A   7.5  6.0   10  6.75
1  Bình   A   8.0  7.5   12  7.75
2   Chi   B   6.5  8.0    8  7.25
3  Dũng   B   9.0  8.5   15  8.75
4    Hà   A   6.0  6.5    5  6.00
Bảng gốc vẫn giữ điểm toán của Hà: 5.5
```

- `rename`: đổi tên cột (từ điển `cũ: mới`); `drop`: xóa cột; `.loc[dòng, cột] = giá_trị`: sửa một ô.
- `rename`, `drop` **không** sửa bảng gốc mà trả về bảng mới.

Lưu ý

Cột `tb` của Hà vẫn 6,00 vì được tính **trước** khi sửa điểm – pandas không tự cập nhật; phải tính lại.

Gom nhóm và đếm: `groupby`, `value_counts`

ch02.py — ô pd_groupby

```
print(bang.groupby("lop")[["toan", "ly",
    "tb"]].mean().round(2))
print(bang["xep_loai"].value_counts())
```

Kết quả

```
      toan    ly    tb
lop
A      7.0  6.67  6.83
B      8.0  7.83  7.92
xep_loai
Khá            3
Trung bình     2
Giỏi           1
Name: count, dtype: int64
```

- `groupby("lop")`: **gom nhóm** (*group by*) – chia bảng theo giá trị của cột `lop`, rồi tính toán trên từng nhóm. Giống *PivotTable* trong Excel.
- Kết quả: lớp B có điểm trung bình cao hơn lớp A ở cả hai môn.
- `value_counts()`: **đếm tần suất** (*frequency count*) mỗi giá trị – rất hữu ích để xem phân bố của **nhãn**.

Bài tập mẫu: tính tay `groupby`

Kiểm tra bằng tay điểm toán trung bình của lớp A và lớp B.

Bài tập mẫu: Chia – tính – ghép *(split – apply – combine)*

Lời giải.

1. **Chia nhóm**: lớp A = An (7,5), Bình (8,0), Hà (5,5); lớp B = Chi (6,5), Dũng (9,0), Khoa (8,5).
2. **Tính trên từng nhóm**: A: $21/3 = 7{,}0$; B: $24/3 = 8{,}0$.
3. **Ghép kết quả** thành bảng, mỗi nhóm một dòng.

Mọi lệnh `groupby` đều hoạt động theo ba bước này.

Nhiều thống kê cho mỗi nhóm: `agg`

ch02.py — ô pd_groupby_nhieu

```
# Mỗi lớp: có bao nhiêu SV, điểm toán cao nhất, giờ tự học trung bình
print(bang.groupby("lop").agg(so_sv=("ten", "count"),
                              toan_max=("toan", "max"),
                              gio_tb=("gio_tu_hoc", "mean")).round(2))
```

Kết quả

```
     so_sv  toan_max  gio_tb
lop
A        3       8.0    9.00
B        3       9.0   11.33
```

Mỗi tham số có dạng `tên_cột_mới=("cột_gốc", "phép_tính")`: đếm số sinh viên, điểm toán cao nhất, giờ tự học trung bình của mỗi lớp.

Ví dụ tổng hợp: phân tích hóa đơn nhà hàng (tips)

Dữ liệu **tips** của Seaborn: 244 hóa đơn – tổng tiền `total_bill`, tiền boa `tip`, giới tính, hút thuốc, ngày, bữa, số người `size`. Bốn câu hỏi, bốn kiểu lệnh:

ch02.py — ô pd_tong_hop_tips

```python
# Ví dụ tổng hợp trên dữ liệu tips (244 hóa đơn của một nhà hàng)
tips = sns.load_dataset("tips")
print("Số hóa đơn mỗi ngày:")
print(tips["day"].value_counts())
print("Hóa đơn trung bình theo bữa:")
print(tips.groupby("time", observed=True)["total_bill"].mean().round(2))
print("3 hóa đơn lớn nhất:")
print(tips.nlargest(3, "total_bill")[["total_bill", "tip", "day", "size"]])
ban_dong = tips[(tips["size"] >= 4) & (tips["day"] == "Sun")]
print("Số bàn từ 4 người vào Chủ nhật:", len(ban_dong))
```

Ví dụ tổng hợp: đọc kết quả

Kết quả

```
Số hóa đơn mỗi ngày:
day
Sat     87
Sun     76
Thur    62
Fri     19
Name: count, dtype: int64
Hóa đơn trung bình theo bữa:
time
Lunch     17.17
Dinner    20.80
Name: total_bill, dtype: float64
3 hóa đơn lớn nhất:
     total_bill    tip  day  size
170       50.81  10.00  Sat     3
212       48.33   9.00  Sat     4
59        48.27   6.73  Sat     4
Số bàn từ 4 người vào Chủ nhật: 22
```

- `value_counts`: thứ Bảy đông nhất.
- `groupby`: bữa tối cao hơn trưa $\approx$ 3,6 đô.
- `nlargest(3, ...)`: 3 hóa đơn lớn nhất – đều thứ Bảy.
- Lọc hai điều kiện: 22 bàn ≥ 4 người vào Chủ nhật.

Mẹo nhỏ

Đặt câu hỏi bằng tiếng Việt trước, rồi “dịch” sang lệnh pandas.

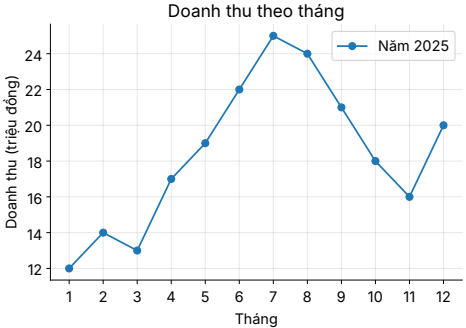
Phần 2

Trực quan hóa dữ liệu

Biểu đồ đường với Matplotlib

ch02.py — ô plt_line

```
thang = np.arange(1, 13)
doanh_thu = np.array([12, 14, 13, 17, 19, 22, 25, 24,
    ↪ 21, 18, 16, 20])
plt.plot(thang, doanh_thu, marker="o",
    ↪ color="tab:blue", label="Năm 2025")
plt.xlabel("Tháng")
plt.ylabel("Doanh thu (triệu đồng)")
plt.title("Doanh thu theo tháng")
plt.xticks(thang)
plt.legend()
plt.show()
```



- Quy ước: `import matplotlib.pyplot as plt`.
- Khuôn chung của mọi biểu đồ: **vẽ** (*plot*) → **đặt tên trục** (*xlabel, ylabel*) → **tiêu đề** (*title*) → `plt.show()`.
- **Biểu đồ đường** (*line chart*): xem **xu hướng theo thời gian** – ở đây là doanh thu theo tháng.

Dữ liệu tips: 244 hóa đơn của một nhà hàng

ch02.py — ô tips_head

```
print(tips.head())
print(tips.shape)
```

Kết quả

```
   total_bill   tip     sex smoker  day    time  size
0       16.99  1.01  Female     No  Sun  Dinner     2
1       10.34  1.66    Male     No  Sun  Dinner     3
2       21.01  3.50    Male     No  Sun  Dinner     3
3       23.68  3.31    Male     No  Sun  Dinner     2
4       24.59  3.61  Female     No  Sun  Dinner     4
(244, 7)
```

`total_bill`	tổng hóa đơn (USD)	`day`	ngày trong tuần
`tip`	tiền boa (*tip*)	`time`	bữa trưa (*Lunch*) / tối (*Dinner*)
`sex`, `smoker`	giới tính, có hút thuốc	`size`	số người trong bàn

Có sẵn trong Seaborn: `sns.load_dataset("tips")` (cần Internet ở lần đầu).

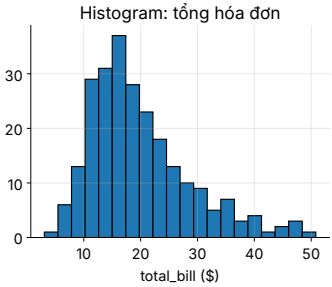
Nhiều biểu đồ trong một hình: `plt.subplots`

ch02.py — ô plt_subplots

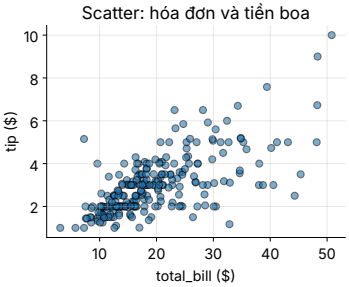
```python
import seaborn as sns
tips = sns.load_dataset("tips")      # dữ liệu tiền boa ở một nhà hàng (244 hóa đơn)
fig, ax = plt.subplots(1, 3, figsize=(13, 3.8))
ax[0].hist(tips["total_bill"], bins=20, color="tab:blue", edgecolor="k")
ax[0].set_title("Histogram: tổng hóa đơn")
ax[0].set_xlabel("total_bill ($)")
ax[1].scatter(tips["total_bill"], tips["tip"], alpha=0.6, edgecolors="k")
ax[1].set_title("Scatter: hóa đơn và tiền boa")
ax[1].set_xlabel("total_bill ($)"); ax[1].set_ylabel("tip ($)")
dem_ngay = tips["day"].value_counts().sort_index()
ax[2].bar(dem_ngay.index.astype(str), dem_ngay.values, color="tab:orange", edgecolor="k")
ax[2].set_title("Bar: số hóa đơn theo ngày")
plt.tight_layout()
plt.show()
```

- `fig, ax = plt.subplots(1, 3, figsize=...)`: tạo một hình với **1 dòng, 3 ô** vẽ; `ax[0]`, `ax[1]`, `ax[2]` là ba ô.
- Mỗi ô có lệnh vẽ riêng: `ax[0].hist`, `ax[1].scatter`, `ax[2].bar`; `plt.tight_layout()` tự căn khoảng cách.

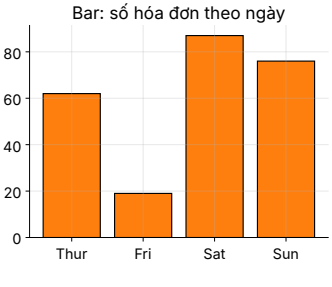
Kết quả: ba loại biểu đồ cơ bản



Histogram (*tần suất*)
phân bố của một cột số: hóa đơn phổ biến 10–20 USD, lệch phải



Scatter (*phân tán*)
quan hệ giữa hai cột số: hóa đơn lớn thì tiền boa lớn

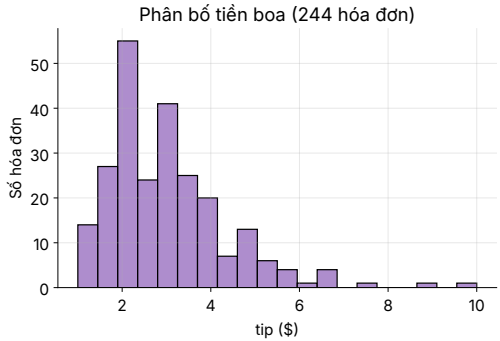


Bar (*cột*)
so sánh giữa các nhóm: số hóa đơn theo ngày

Histogram với Seaborn

ch02.py — ô sns_hist

```
1  plt.figure(figsize=(6.4, 3.8))
2  sns.histplot(data=tips, x="tip", bins=20, color="tab:purple")
3  plt.title("Phân bố tiền boa (244 hóa đơn)")
4  plt.xlabel("tip ($)")
5  plt.ylabel("Số hóa đơn")
6  plt.show()
```



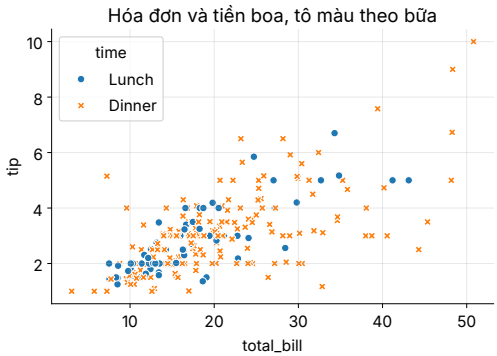
- **Histogram** (*biểu đồ tần suất*): chia trục thành các khoảng (*bins*), đếm số giá trị rơi vào mỗi khoảng.
- Tiền boa phổ biến nhất khoảng 2–3,5 đô; vài lần boa 7–10 đô (đuôi dài bên phải – **lệch phải** (*right-skewed*)).
- Các "cột" nhô cao ở số tròn 2, 3, 4 đô: khách hay boa số tròn – chỉ nhìn biểu đồ mới thấy.

Biểu đồ phân tán tô màu theo nhóm

ch02.py — ô sns_scatter

```
plt.figure(figsize=(6.4, 4))
sns.scatterplot(data=tips, x="total_bill", y="tip", hue="time",
    ↪ style="time")
plt.title("Hóa đơn và tiền boa, tô màu theo bữa")
plt.show()
```

- `hue`: tô màu, `style`: đổi hình dạng điểm theo một cột chữ.
- Bữa tối (cam) trải rộng sang phải: nhiều hóa đơn lớn.
- Cùng mức hóa đơn, hai màu xen kẽ: bữa ăn không làm đổi **tỉ lệ** boa nhiều.



Chọn loại biểu đồ theo mục đích

Biểu đồ	Dùng để	Lệnh
Đường (*line chart*)	Xem xu hướng theo thời gian	`plt.plot`
Tần suất (*histogram*)	Xem *phân bố* của một cột số: giá trị nào phổ biến, có lệch không	`plt.hist`, `sns.histplot`
Phân tán (*scatter plot*)	Xem *quan hệ* giữa hai cột số	`plt.scatter`
Cột (*bar chart*)	So sánh giá trị giữa các nhóm	`plt.bar`, `sns.countplot`
Hộp (*box plot*)	So sánh phân bố giữa các nhóm, phát hiện ngoại lai	`sns.boxplot`
Nhiệt (*heatmap*)	Hiển thị ma trận số, ví dụ ma trận tương quan	`sns.heatmap`

Câu hỏi cho lớp

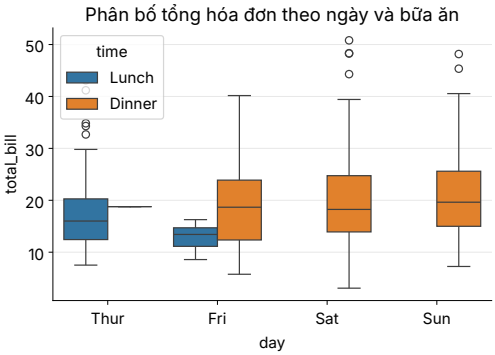
Muốn biết cột `tuoi` có giá trị bất thường không, nên vẽ gì? Còn muốn xem quan hệ giữa `tuoi` và `thu_nhap`?

Biểu đồ hộp với Seaborn

ch02.py — ô sns_box

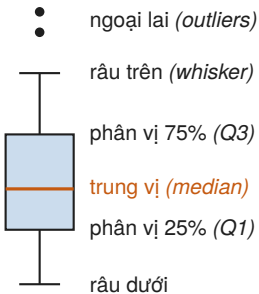
```
1 plt.figure(figsize=(6.4, 4))
2 sns.boxplot(data=tips, x="day", y="total_bill",
    ↪ hue="time")
3 plt.title("Phân bố tổng hóa đơn theo ngày và bữa
    ↪ ăn")
4 plt.show()
```

Seaborn làm việc trực tiếp với DataFrame: chỉ cần cho tên cột trục `x`, trục `y` và cột tô màu `hue`.



Bữa tối (cam) nhìn chung cao hơn bữa trưa; thứ Bảy, Chủ nhật có vài hóa đơn rất lớn.

Đọc biểu đồ hộp



Toán dễ hiểu: Năm con số của một "hộp"

- Vạch giữa hộp: **trung vị** (*median*) – giá trị ở chính giữa khi sắp xếp.
- Cạnh dưới, cạnh trên: **phân vị** (*percentile*) 25% và 75% – **một nửa** số dữ liệu nằm trong hộp.
- Hai "râu": kéo tới giá trị nhỏ nhất và lớn nhất **không bị coi là ngoại lai**.
- Chấm tròn riêng lẻ: **giá trị ngoại lai** (*outlier*) – những giá trị khác thường.

Hộp **càng cao** = dữ liệu **càng phân tán**. Quy tắc xác định ngoại lai (1,5 IQR): Chương 3.

Bài tập mẫu: tính tay năm con số của một hộp

Tiền boa (đô) của 7 bàn: **1, 2, 2, 3, 3, 4, 10**. Tìm trung vị, Q_1, Q_3; có ngoại lai không? (Ngoại lai: ngoài $[Q_1 - 1{,}5\,\text{IQR};\ Q_3 + 1{,}5\,\text{IQR}]$, $\text{IQR} = Q_3 - Q_1$.)

Bài tập mẫu

Lời giải.

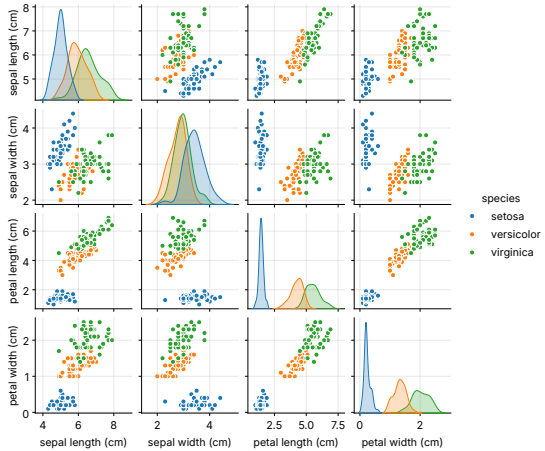
- 7 số đã sắp xếp $\Rightarrow$ **trung vị** (*median*) là số thứ 4: **3**.
- Q_1 ở vị trí $(7-1) \times 0{,}25 = 1{,}5$ (đếm từ 0): giữa 2 và 2 $\Rightarrow Q_1 = 2$.
- Q_3 ở vị trí $(7-1) \times 0{,}75 = 4{,}5$: giữa 3 và 4 $\Rightarrow Q_3 = 3{,}5$.
- $\text{IQR} = 1{,}5$; ngưỡng trên $3{,}5 + 2{,}25 = 5{,}75$; ngưỡng dưới $2 - 2{,}25 = -0{,}25$.
- 10 vượt ngưỡng trên $\Rightarrow$ **ngoại lai** (chấm riêng); râu trên kéo tới 4, râu dưới tới 1.

Biểu đồ cặp (pairplot) với Iris

ch02.py — ô sns_pair

```
1 from sklearn.datasets import load_iris
2 iris_df = load_iris(as_frame=True).frame
3 iris_df["species"] =
    ↪ iris_df["target"].map({0: "setosa", 1:
    ↪ "versicolor", 2: "virginica"})
4 sns.pairplot(iris_df.drop(columns="target"),
    ↪ hue="species", height=1.9)
5 plt.show()
```

- **Biểu đồ cặp** (*pair plot*): vẽ phân tán cho **mọi cặp** đặc trưng; đường chéo là phân bố của từng đặc trưng.
- Rất hữu ích khi **khám phá dữ liệu** (*exploratory data analysis, EDA*) có ít cột.
- Hai đặc trưng `petal` tách 3 loài rõ nhất; `sepal width` gần như không giúp phân biệt ⇒ gợi ý về **chọn đặc trưng** (*feature selection*).



Đọc biểu đồ cặp

- Hình gồm $4 \times 4 = 16$ ô; ô dòng i, cột j là biểu đồ phân tán giữa đặc trưng i và đặc trưng j.
- Đường chéo (đặc trưng với chính nó): đường **phân bố** (*distribution*) của đặc trưng đó cho từng loài.
- Ô dòng 3, cột 4 chính là biểu đồ phân tán cánh hoa ở Chương 1.
- Hai đặc trưng **cánh hoa** tách ba loài rõ nhất; `sepal width`: ba đường phân bố chồng lên nhau $\Rightarrow$ gần như không giúp phân biệt.

Mẹo nhỏ

Một bức hình đã gợi ý **nên chọn đặc trưng nào** – trước khi huấn luyện bất kỳ mô hình nào.

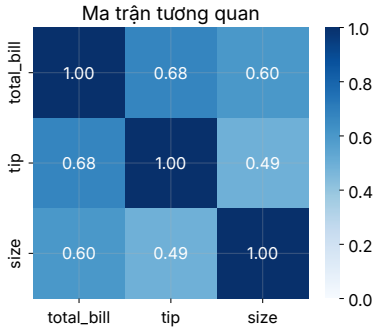
Ma trận tương quan và biểu đồ nhiệt

ch02.py — ô sns_heatmap

```
1 tuong_quan = tips[["total_bill", "tip", "size"]].corr()
2 print(tuong_quan.round(3))
3 plt.figure(figsize=(4.8, 3.8))
4 sns.heatmap(tuong_quan, annot=True, fmt=".2f",
    ↪ cmap="Blues", vmin=0, vmax=1)
5 plt.title("Ma trận tương quan")
6 plt.show()
```

Kết quả

```
             total_bill    tip   size
total_bill        1.000  0.676  0.598
tip               0.676  1.000  0.489
size              0.598  0.489  1.000
```



- **Ma trận tương quan** (*correlation matrix*): mỗi ô cho biết hai cột "đi cùng nhau" mạnh đến đâu, từ -1 đến 1.
- **Biểu đồ nhiệt** (*heatmap*) tô màu ma trận: ô càng đậm, quan hệ càng mạnh. `total_bill` và `tip` tương quan khá mạnh (0,68).
- Hệ số tương quan được giải thích ở Chương 3.

Phần 3

Làm quen với scikit-learn

Các bộ dữ liệu có sẵn

ch02.py — ô sk_datasets

```
from sklearn import datasets
for ten in ["load_iris", "load_wine", "load_breast_cancer", "load_digits", "load_diabetes"]:
    du_lieu = getattr(datasets, ten)()
    print(f"{ten:20s} X: {str(du_lieu.data.shape):10s} y: {du_lieu.target.shape}")
```

Kết quả

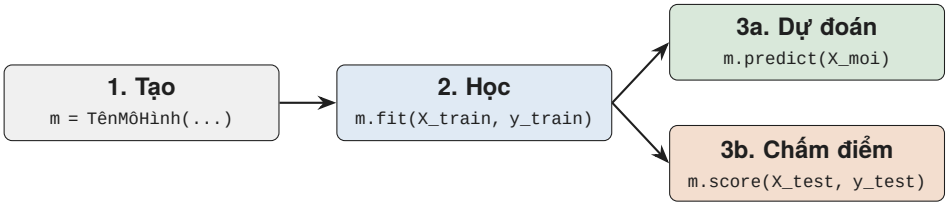
```
load_iris            X: (150, 4)   y: (150,)
load_wine            X: (178, 13)  y: (178,)
load_breast_cancer   X: (569, 30)  y: (569,)
load_digits          X: (1797, 64) y: (1797,)
load_diabetes        X: (442, 10)  y: (442,)
```

`scikit-learn` đóng gói sẵn nhiều **bộ dữ liệu mẫu** (*toy datasets*), nạp tức thì mà không cần Internet – đó là lý do giáo trình dùng chúng làm ví dụ xuyên suốt.

Các bộ dữ liệu dùng trong giáo trình

Bộ dữ liệu	Cách nạp	Nội dung và bài toán
Iris	`load_iris`	150 bông hoa, 4 đặc trưng; **phân loại** 3 loài
Wine	`load_wine`	178 mẫu rượu, 13 chỉ số hóa học; **phân loại** 3 giống nho
Breast Cancer	`load_breast_cancer`	569 khối u, 30 đặc trưng; **phân loại** lành/ác tính
Digits	`load_digits`	1797 ảnh chữ số 8×8; **phân loại** 10 chữ số
Diabetes	`load_diabetes`	442 bệnh nhân, 10 chỉ số; **hồi quy** mức tiến triển bệnh
Auto MPG	`sns.load_dataset("mpg")`	398 mẫu xe; **hồi quy** mức tiêu hao nhiên liệu
Titanic, Tips, Penguins	`sns.load_dataset`	Dữ liệu bảng có cột chữ, có ô trống (cần Internet)

Cách gọi chung của mọi mô hình



- Điểm mạnh lớn nhất của `scikit-learn`: mọi thuật toán là một **bộ ước lượng** (*estimator*) dùng **chung một cách gọi** (*consistent API*).
- Biết dùng **một** mô hình là biết dùng **tất cả**.
- Với mô hình phân loại, `.score()` tính sẵn **độ chính xác** – không cần gọi `predict` rồi `accuracy_score` như Chương 1.

Ba thuật toán – cùng một cách gọi

ch02.py — ô sk_nhieu_mo_hinh

```python
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.tree import DecisionTreeClassifier
from sklearn.linear_model import LogisticRegression

X, y = datasets.load_iris(return_X_y=True)          # trả về thẳng X, y dạng mảng NumPy
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42, stratify=y)

cac_mo_hinh = {
    "kNN (k=5)": KNeighborsClassifier(n_neighbors=5),
    "Cây quyết định": DecisionTreeClassifier(random_state=42),
    "Hồi quy Logistic": LogisticRegression(max_iter=1000),
}
for ten, mo_hinh in cac_mo_hinh.items():
    mo_hinh.fit(X_train, y_train)                    # 1. học
    diem_so = mo_hinh.score(X_test, y_test)          # 2. chấm điểm (accuracy)
    print(f"{ten:18s}: accuracy = {diem_so:.4f}")
```

Kết quả

```
kNN (k=5)          : accuracy =
↪  0.9778
Cây quyết định     : accuracy =
↪  0.9333
Hồi quy Logistic   : accuracy =
↪  0.9333
```

- Chưa cần hiểu các thuật toán: hãy để ý `fit` và `score` **giống hệt nhau** cho cả ba mô hình.
- `load_iris(return_X_y=True)` trả thẳng về cặp `X, y` dạng mảng NumPy.

Ví dụ đời thường: ổ cắm điện tiêu chuẩn

Ví dụ đời thường: Bên trong khác nhau, "phích cắm" giống nhau

Quạt, tủ lạnh, máy giặt hoạt động khác nhau hoàn toàn bên trong, nhưng đều cắm vào cùng một loại ổ điện. `scikit-learn` thiết kế mọi mô hình như vậy: thuật toán bên trong khác nhau, nhưng "phích cắm" `fit`, `predict`, `score` giống hệt nhau.

- Đổi thuật toán chỉ là **đổi một dòng** tạo mô hình.
- Ba mô hình trên Iris đều đạt trên 93% – Iris là bài toán dễ.
- `.score()` của mô hình phân loại tính sẵn accuracy.

Bộ biến đổi: `fit` rồi `transform`

ch02.py — ô sk_transformer

```
from sklearn.preprocessing import MinMaxScaler
du_lieu_nho = np.array([[1.0, 100.0],
                        [2.0, 300.0],
                        [3.0, 500.0]])
bo_bien_doi = MinMaxScaler()              # đưa mỗi cột về khoảng [0, 1]
bo_bien_doi.fit(du_lieu_nho)              # học min, max của từng cột
print("min mỗi cột:", bo_bien_doi.data_min_, " max mỗi cột:", bo_bien_doi.data_max_)
print(bo_bien_doi.transform(du_lieu_nho))
```

Kết quả

```
min mỗi cột: [  1. 100.]  max mỗi cột: [  3. 500.]
[[0.  0. ]
 [0.5 0.5]
 [1.  1. ]]
```

Ngoài mô hình dự đoán, `scikit-learn` còn có các **bộ biến đổi** (*transformer*) dùng để tiền xử lý:

- `fit`: **học** thông tin từ dữ liệu – ở đây `MinMaxScaler` học giá trị min, max của từng cột.
- `transform`: **biến đổi** dữ liệu bằng thông tin đã học – co giãn mỗi cột về khoảng $[0, 1]$.
- `fit_transform`: làm cả hai bước một lần. Vì sao cần co giãn? $\Rightarrow$ Chương 4.

Bài tập mẫu: tính tay MinMaxScaler

Công thức cho mỗi cột: $x' = \dfrac{x - \min}{\max - \min}$. Kiểm tra kết quả, rồi biến đổi dòng mới $(4{,}0;\ 200)$.

Bài tập mẫu

Lời giải.

- Cột 1 (min $= 1$, max $= 3$): $1 \to 0$; $2 \to 0{,}5$; $3 \to 1$.
- Cột 2 (min $= 100$, max $= 500$): $100 \to 0$; $300 \to 200/400 = 0{,}5$; $500 \to 1$.
- Dòng mới dùng **lại** min, max đã học (không học lại): $(4-1)/2 = 1{,}5$ và $(200-100)/400 = 0{,}25$.

1,5 nằm ngoài $[0, 1]$ vì 4,0 lớn hơn mọi giá trị lúc `fit` – hoàn toàn bình thường. Quy tắc "`fit` một lần trên dữ liệu huấn luyện, `transform` cho mọi dữ liệu" rất quan trọng ở Chương 4.

Bảng thuật ngữ Việt – Anh của buổi học

Tiếng Việt	Tiếng Anh
Dữ liệu dạng bảng	Tabular data
Chuỗi có nhãn	Series
Khung dữ liệu	DataFrame
Chỉ mục	Index
Tệp giá trị phân cách dấu phẩy	CSV file
Giá trị thiếu	Missing value (NaN)
Lọc dòng	Filtering
Gom nhóm	Group by
Tổng hợp	Aggregation

Tiếng Việt	Tiếng Anh
Trực quan hóa	Visualization
Biểu đồ tần suất	Histogram
Biểu đồ phân tán	Scatter plot
Biểu đồ hộp	Box plot
Giá trị ngoại lai	Outlier
Trung vị	Median
Ma trận tương quan	Correlation matrix
Mô hình ước lượng	Estimator
Bộ biến đổi	Transformer

Phần 4

Thực hành và tổng kết

Thực hành 2.1: Khách hàng nào boa nhiều hơn? – Bước 1

Bối cảnh. Chủ nhà hàng muốn biết khách boa thế nào để sắp xếp nhân viên phục vụ. Vì hóa đơn lớn thì tiền boa thường lớn, ta so sánh bằng **tỉ lệ boa** (% so với hóa đơn) thay vì số tiền boa.

ch02.py — ô lab_buoc1

```
1 tips = sns.load_dataset("tips")
2 tips["ti_le_boa"] = tips["tip"] /
    ↪ tips["total_bill"] * 100   # % tiền boa
3 print(tips[["total_bill", "tip",
    ↪ "ti_le_boa"]].describe().round(2))
```

Kết quả

	total_bill	tip	ti_le_boa
count	244.00	244.00	244.00
mean	19.79	3.00	16.08
std	8.90	1.38	6.11
min	3.07	1.00	3.56
25%	13.35	2.00	12.91
50%	17.80	2.90	15.48
75%	24.13	3.56	19.15
max	50.81	10.00	71.03

- Tạo cột **đặc trưng mới** `ti_le_boa` từ hai cột cũ – một ví dụ **tạo đặc trưng** (*feature engineering*).
- Trung bình khách boa khoảng **16%**; giá trị lớn nhất 71% bất thường (**ngoại lai**), có thể do hóa đơn rất nhỏ.

Thực hành 2.1: Bước 2 – so sánh theo nhóm

ch02.py — ô lab_buoc2

```
print(tips.groupby("day",
    ↪ observed=True)["ti_le_boa"].mean().round(2))
print(tips.groupby("smoker",
    ↪ observed=True)["ti_le_boa"].mean().round(2))
```

Kết quả

```
day
Thur    16.13
Fri     16.99
Sat     15.32
Sun     16.69
Name: ti_le_boa, dtype: float64
smoker
Yes    16.32
No     15.93
Name: ti_le_boa, dtype: float64
```

- Tỉ lệ boa giữa các ngày chênh nhau không nhiều (15–17%).
- Người hút thuốc và không hút thuốc boa gần như nhau.
- `observed=True` chỉ để tránh cảnh báo khi cột nhóm có kiểu **category** (*categorical dtype*).

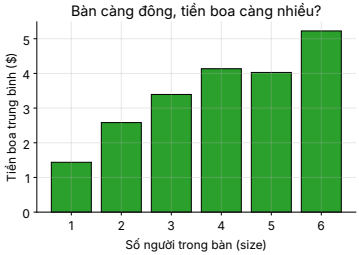
Thực hành 2.1: Bước 3 – bàn càng đông, tiền boa càng nhiều?

ch02.py — ô lab_buoc3

```
tb_size = tips.groupby("size")["tip"].mean()
plt.figure(figsize=(6, 3.6))
plt.bar(tb_size.index, tb_size.values,
    ↪ color="tab:green", edgecolor="k")
plt.xlabel("Số người trong bàn (size)")
plt.ylabel("Tiền boa trung bình ($)")
plt.title("Bàn càng đông, tiền boa càng nhiều?")
plt.show()
print(tb_size.round(2))
```

Kết quả

```
size
1    1.44
2    2.58
3    3.39
4    4.14
5    4.03
6    5.22
Name: tip, dtype: float64
```



Nhận xét. Bàn càng đông, tiền boa càng nhiều; bàn 5 người là ngoại lệ vì có **rất ít** bàn 5 người (con số **kém tin cậy**). Nên ưu tiên nhân viên giàu kinh nghiệm cho bàn đông.

Lỗi thường gặp

1. **Chạy ô sau trước ô trước** trên Colab: lỗi `NameError: name 'np' is not defined` nghĩa là ô `import` chưa chạy. Chạy các ô theo thứ tự từ trên xuống.
2. **Đưa mảng một chiều vào `scikit-learn`**: lỗi `Expected 2D array, got 1D array` $\Rightarrow$ dùng `X.reshape(-1, 1)` hoặc chọn cột bằng `df[["cot"]]`.
3. **Nhầm `axis=0` và `axis=1`**: luôn kiểm tra `shape` của kết quả – muốn một giá trị cho mỗi cột thì số phần tử phải bằng số cột.
4. **Dùng `and`/`or` khi lọc trong pandas**: phải dùng `&`/`|` và bọc từng điều kiện trong ngoặc tròn.
5. **Nhầm `.loc` và `.iloc`**: sau khi lọc hoặc sắp xếp, chỉ mục không còn liên tiếp; `.loc[0]` và `.iloc[0]` có thể là hai dòng khác nhau.
6. **Quên `plt.show()` hoặc vẽ chồng hình**: thiếu `plt.figure()` giữa các lệnh vẽ sẽ vẽ chồng lên nhau.

Tóm tắt

- **NumPy**: lưu dữ liệu dạng mảng; luôn kiểm tra `shape`. Phép toán **theo phần tử** (*element-wise*) và **broadcasting** (*lan truyền kích thước*) giúp viết công thức không cần vòng lặp.
- `axis=0`: tính theo cột (mỗi cột một kết quả); `axis=1`: tính theo dòng.
- **pandas**: `read_csv`, `head`, `info`, `describe`; chọn cột bằng `[]`, chọn dòng bằng `.loc`/`.iloc` hoặc mặt nạ; tổng hợp bằng `groupby`, `value_counts`.
- **Matplotlib/Seaborn**: histogram – phân bố; scatter – quan hệ; boxplot – so sánh nhóm; heatmap – tương quan; pairplot – mọi cặp đặc trưng.
- **scikit-learn**: **mô hình** (*estimator*) dùng `fit` → `predict`/`score`; **bộ biến đổi** (*transformer*) dùng `fit` → `transform`.

Câu hỏi thảo luận

1. Phân biệt Series và DataFrame. `df["a"]` và `df[["a"]]` khác nhau thế nào?
2. Khi nào dùng `.loc`, khi nào dùng `.iloc`?
3. Muốn biết cột `tuoi` có giá trị bất thường không, nên vẽ biểu đồ gì? Còn muốn xem quan hệ giữa `tuoi` và `thu_nhap`?
4. Trình bày ba bước dùng một mô hình trong scikit-learn. Bộ biến đổi khác mô hình dự đoán ở điểm nào?

Bài tập Buổi 2.2

- ⋆ **2.6**: Với bảng `bang`: (a) sinh viên tự học ít hơn 10 giờ; (b) thêm cột `dat`; (c) giờ tự học trung bình mỗi lớp.
- ⋆⋆ **2.7**: tips: bữa trưa hay tối hóa đơn cao hơn? Histogram `tip` 25 khoảng; `sns.scatterplot` tô màu theo `time`.
- ⋆⋆ **2.8**: Penguins: số dòng, cột, kiểu dữ liệu, ô trống mỗi cột; pairplot theo loài.
- ⋆⋆ **2.9**: Wine: trung bình `alcohol`, `color_intensity` theo lớp; boxplot `proline` theo lớp.
- ⋆ **2.10**: Tiền boa 9 bàn 2, 3, 3, 4, 2, 5, 3, 12, 4: tính tay trung vị, Q_1, Q_3, ngoại lai; kiểm tra bằng pandas.
- ⋆⋆⋆ **2.11**: Viết hàm `tom_tat(df, cot_nhom, cot_so)` trả về số dòng, trung bình, nhỏ nhất, lớn nhất theo nhóm.

Chi tiết: *Bài tập Buổi 2.2*, Chương 2. Buổi sau: **Chương 3 – Toán cần thiết cho học máy**.